

Experiment 9

AIM: Implement the Private Ethereum Blockchain using Geth

THEORY:

Q1: What is Geth?

Geth (Go-Ethereum) is one of the most widely used Ethereum clients, written in the Go programming language. It allows users to run an Ethereum node, mine Ether, and interact with the blockchain through JSON-RPC, command-line, or JavaScript consoles.

Q2: Significance of a Private Ethereum Network

Setting up a Private Ethereum Network involves creating a customized blockchain environment

isolated from the public Ethereum network. It enables developers to test smart contracts,

consensus mechanisms, and network behavior without incurring gas costs or depending on

public validators.

Q3: Steps for creating a Private Ethereum Network

Steps Involved:

- **Choosing a Network ID:**

Each private blockchain must have a unique network ID to differentiate it from public or other private networks.

- **Choosing a Consensus Algorithm:**

Ethereum supports different consensus mechanisms such as Proof of Work (PoW) and Proof of Authority (PoA). In private networks, Clique (PoA) is often used for faster block confirmation.

- **Creating a Genesis Block:**

The genesis block is the first block of the blockchain, defining network parameters such as difficulty, gas limit, initial account balances, and consensus settings.

- **Initializing the Geth Database:**

The Geth init command initializes the blockchain using the genesis.json file, preparing the data directory for node operation.

- **Setting up Networking:**

Nodes communicate using enodes (Ethereum Node URLs) that contain IP addresses and public keys for peer discovery and connection.

- **Running the Member Nodes:**

Each participant runs a Geth node that connects to the private network and participates in block validation and transaction processing.

- **Running a Signer (in Clique):**

In the Clique consensus, designated signers validate and sign blocks. These accounts are pre-configured in the genesis file to ensure trusted block production.

NOTE: Download the genesis file and edit the account details, including the public keys of all participating peers in the network.

PROCEDURE

Step 1: Install Geth

```
wget
```

```
https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
```

```
tar -xvf geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
```

```
mkdir -p ~/bin
```

```
cp geth-linux-amd64-1.13.14-2bd6bd01/geth ~/bin/geth113
```

export

PATH="\$HOME/bin:\$PATH"

```
Welcome to Cloud Shell! Type "help" to get started, or type "pswd" to try prompting with Secure Chat.
To set your Cloud Platform project in this session use "gcloud config set project [PROJECT_ID]".
You can view your projects by running "gcloud projects list".
g2022_aditi_taksale@cloudshell:~$ wget
https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
tar -xvf geth-linux-amd64-1.13.14-2bd6bd01.tar.gz mkdir -p ~/bin
cp geth-linux-amd64-1.13.14-2bd6bd01/geth ~/bin/geth113

Resolving gethstore.blob.core.windows.net (gethstore.blob.core.windows.net)... 20.69.60.164
Connecting to gethstore.blob.core.windows.net (gethstore.blob.core.windows.net)[20.69.60.164]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2924813 (2.7M) [application/octet-stream]
Saving to: 'geth-linux-amd64-1.13.14-2bd6bd01.tar.gz.1'

geth-linux-amd64-1.13.14-2bd6bd01.tar.gz.1  100%[=====] 26.94M  4.90MB/s   16.91s

2024-04-08 17:25:18 13.31 MB/s ~ 'geth-linux-amd64-1.13.14-2bd6bd01.tar.gz.1' saved [2924813/2924813]

geth-linux-amd64-1.13.14-2bd6bd01/
geth-linux-amd64-1.13.14-2bd6bd01/COPYING
geth-linux-amd64-1.13.14-2bd6bd01/geth
geth
Version: 1.13.14-stable
Git Commit: 2bd6bd01c2ef561ed77fc21b431f6a38ac14427a1
Git Commit Date: 20240227
Architecture: amd64
Go Version: go1.21.4
Operating System: linux
GORATHU=/home/g2022_ria_chaudhari/gopath:/google/gopath
GIMCOPF=
```

geth113 version

Step 2: Create folders and accounts for both nodes

```
g2022_aditi_taksale@cloudshell:~$ mkdir -p ~/eth-private/node1 ~/eth-private/node2
g2022_aditi_taksale@cloudshell:~$ geth113 --datadir ~/eth-private/node1 account new
INFO [04-0817:27:46.537] Maximum peer count      ETH=50 total=50
INFO [04-0817:27:46.540] Smartcard socket not found, disabling  err="stat /run/pcscd/pcscd.comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not forget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key: 0aC8r5b123bb5a223c4bcb4ca8363ed92a2f9878
Path of the secret key file: /home/g2022_ria_chaudhari/eth-private/node1/keystore/UTC--2024-04-08T17-27-52.865021965Z--c8fe5b123bb5a223c4bcb4ca8363ed92a2f9878

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!
```

```
g2022_aditi_taksale@cloudshell:~$ geth113 --datadir ~/eth-private/node2 account new
INFO [04-0817:28:38.211] Maximum peer count      ETH=50 total=50
INFO [04-0817:28:38.214] Smartcard socket not found, disabling  err="stat /run/pcscd/pcscd.comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not forget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key: 0x41911d883221851rc0C0r7c2a3f623d507bb19
Path of the secret key file: /home/g2022_ria_chaudhari/eth-private/node2/keystore/UTC--2024-04-08T17-28-43.340511622Z--41911d883221851dd8b0fe7c2a3f623d507bb19

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!
```

Step 3: Create password files

```
g2022_aditi_taksale@cloudshell:~$ echo "1234" > ~/eth-private/node1/password.txt
echo "1234" > ~/eth-private/node2/password.txt
```

Step 4: Create genesis.json and verify it looks correct

NODE1="PASTE_NODE1_ADDRESS_WITHOUT_0x_HERE"

NODE2="PASTE_NODE2_ADDRESS_WITHOUT_0x_HERE"

```
cat > ~/eth-private/genesis.json << EOF
```

 $\{$

```
"config": {
```

```
"chainId": 1234,
```

```
"homesteadBlock": 0,
```

```
"eip150Block": 0,
```

```
"eip155Block": 0,
```

```
"eip158Block": 0,
```

```
"byzantiumBlock": 0,
```

"constantinopleBlock": 0,

"petersburgBlock": 0,

```
"clique": {
```

```
"period": 5,
```

```
"epoch": 30000
```

}

 $\},$

```
"difficulty": "1",
```

```
"gasLimit": "8000000",
```

```
"extradata":
```

[illegible][illegible][illegible]


```

q2022_aditi_taksale@cloudshell:~$ geth113 --datadir ~/eth-private/node1 init ~/eth-private/genesis.json
geth113 --datadir ~/eth-private/node2 init ~/eth-private/genesis.json
INFO [04-00:17:42:33.075] Maximum peer count                      ETH=>50 http=>50
INFO [04-00:17:42:33.079] Smartcard socket not found, disabling   err="stat /run/pcscd/pcscd.sock: no such file or directory"
INFO [04-00:17:42:33.085] Set global gas cap                      cap=50,000,000
INFO [04-00:17:42:33.086] Initializing the EVM library            backend=gokzg
INFO [04-00:17:42:33.218] Defaulting to pebble as the backing database
INFO [04-00:17:42:33.219] Allocated cache and file handles       database=/home/q2022_via_chaudhari/eth-private/node1/geth/chaindata cache=16.50MiB handles=16
INFO [04-00:17:42:33.306] Opened ancient database                 database=/home/q2022_via_chaudhari/eth-private/node1/geth/chaindata/ancient/chain readonly=false
INFO [04-00:17:42:33.306] State schema set to default            schema=hash
INFO [04-00:17:42:33.301] Writing custom genesis block
INFO [04-00:17:42:33.304] Persisted trie from memory database     nodes=4 size=478.00B time=3.365396ms gcsize=0.00B gptime=0s livenodes=0 livepairs=0.00B
INFO [04-00:17:42:33.340] Successfully wrote genesis state        database=chaindata hash=38d61a..afc19d
INFO [04-00:17:42:33.340] Defaulting to pebble as the backing database
INFO [04-00:17:42:33.340] Allocated cache and file handles       database=/home/q2022_via_chaudhari/eth-private/node1/geth/lightchaindata cache=16.50MiB handles=16
INFO [04-00:17:42:33.400] Opened ancient database                 database=/home/q2022_via_chaudhari/eth-private/node1/geth/lightchaindata/ancient/chain readonly=false
INFO [04-00:17:42:33.400] State schema set to default            schema=hash
INFO [04-00:17:42:33.404] Writing custom genesis block
INFO [04-00:17:42:33.404] Persisted trie from memory database     nodes=4 size=478.00B time=3.273748ms gcsize=0.00B gptime=0s livenodes=0 livepairs=0.00B
INFO [04-00:17:42:33.438] Successfully wrote genesis state        database=lightchaindata hash=38d61a..afc19d
INFO [04-00:17:42:33.515] Maximum peer count                      ETH=>50 http=>50
INFO [04-00:17:42:33.518] Smartcard socket not found, disabling   err="stat /run/pcscd/pcscd.sock: no such file or directory"
INFO [04-00:17:42:33.523] Set global gas cap                      cap=50,000,000
INFO [04-00:17:42:33.524] Initializing the EVM library            backend=gokzg
INFO [04-00:17:42:33.701] Defaulting to pebble as the backing database
INFO [04-00:17:42:33.701] Allocated cache and file handles       database=/home/q2022_via_chaudhari/eth-private/node2/geth/chaindata cache=16.50MiB handles=16
INFO [04-00:17:42:33.758] Opened ancient database                 database=/home/q2022_via_chaudhari/eth-private/node2/geth/chaindata/ancient/chain readonly=false
INFO [04-00:17:42:33.758] State schema set to default            schema=hash
INFO [04-00:17:42:33.758] Writing custom genesis block
INFO [04-00:17:42:33.762] Persisted trie from memory database     nodes=4 size=478.00B time=3.351028ms gcsize=0.00B gptime=0s livenodes=0 livepairs=0.00B
INFO [04-00:17:42:33.798] Successfully wrote genesis state        database=chaindata hash=38d61a..afc19d
INFO [04-00:17:42:33.798] Defaulting to pebble as the backing database
INFO [04-00:17:42:33.798] Allocated cache and file handles       database=/home/q2022_via_chaudhari/eth-private/node2/geth/lightchaindata cache=16.50MiB handles=16
INFO [04-00:17:42:33.853] Opened ancient database                 database=/home/q2022_via_chaudhari/eth-private/node2/geth/lightchaindata/ancient/chain readonly=false
INFO [04-00:17:42:33.853] State schema set to default            schema=hash
INFO [04-00:17:42:33.853] Writing custom genesis block
INFO [04-00:17:42:33.857] Persisted trie from memory database     nodes=4 size=478.00B time=3.139211ms gcsize=0.00B gptime=0s livenodes=0 livepairs=0.00B
INFO [04-00:17:42:33.891] Successfully wrote genesis state        database=lightchaindata hash=38d61a..afc19d
q2022_aditi_taksale@cloudshell:~$

```

Step 6: Run Node 1 (signer)

```

q2022_aditi_taksale@cloudshell:~$ geth113 --datadir ~/eth-private/node1 \
--networkid 1234 \
--port 30303 \
--http --http.port 8545 \
--http.api personal,eth,net,web3,miner \
--unlock 0x1MOCK1 \
--password ~/eth-private/node1/password.txt \
--mine \
--miner.etherbase 0x1MOCK1 \
--allow-insecure-unlock \
console
INFO [04-00:17:44:52.541] Maximum peer count                      ETH=>50 http=>50
INFO [04-00:17:44:52.548] Smartcard socket not found, disabling   err="stat /run/pcscd/pcscd.sock: no such file or directory"
INFO [04-00:17:44:52.548] Set global gas cap                      cap=50,000,000
INFO [04-00:17:44:52.549] Initializing the EVM library            backend=gokzg
INFO [04-00:17:44:52.700] Allocated trie memory caches           csize=154.00MiB http=254.50MiB
INFO [04-00:17:44:52.701] Using pebble as the backing database    database=/home/q2022_via_chaudhari/eth-private/node1/geth/chaindata cache=16.50MiB handles=16,288
INFO [04-00:17:44:52.730] Opened ancient database                 database=/home/q2022_via_chaudhari/eth-private/node1/geth/chaindata/ancient/chain readonly=false
INFO [04-00:17:44:52.730] State schema set to already existing    schema=hash
INFO [04-00:17:44:52.732] Initializing Ethereum protocol          network=1234 (devchain=0)
INFO [04-00:17:44:52.732] -----
INFO [04-00:17:44:52.733] Chain ID: 1234 (unknown)
INFO [04-00:17:44:52.733] Consensus: Clique (proof-of-authority)
INFO [04-00:17:44:52.733] -----
INFO [04-00:17:44:52.733] Pre-Merge hard forks (block based):
INFO [04-00:17:44:52.733] - Homestead: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/homestead.md)
INFO [04-00:17:44:52.733] - Tangerine Whistle (EIP 150): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/tangerine-whistle.md)
INFO [04-00:17:44:52.733] - Spurious Dragon/1 (EIP 153): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/spurious-dragon.md)
INFO [04-00:17:44:52.733] - Spurious Dragon/2 (EIP 158): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/spurious-dragon.md)
INFO [04-00:17:44:52.733] - Byzantium: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/byzantium.md)
INFO [04-00:17:44:52.733] - Constantinople: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/constantinople.md)
INFO [04-00:17:44:52.733] - Petersburg: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/petersburg.md)
INFO [04-00:17:44:52.733] - Istanbul: #null (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/istanbul.md)
INFO [04-00:17:44:52.733] - Berlin: #null (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/berlin.md)
INFO [04-00:17:44:52.733] - London: #null (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrades/london.md)

```

Step 7: Open new terminal tab and run Node 2 (member)

Step 8: Connect the two nodes In Node 1 console, get its enode

Now, switch to Node 2 console and paste the Node 1 enode

```
> admin.addPeer("enode://37a3db1c1e34eecb3611941ca5b7001168ac8eec28506649e6a5290979c7a78e7277565b37ec5d703871a9970a4794c51bde075d7991723256e0c07c8cb367eb034.21.175.36:30303")
true
```


In Node 2 console, check they are connected

```
> admin.peersINFO [04-08|18:01:18.017] Imported new chain segment      number=196 hash=9ac569..83e85f blocks=1   txs=0 mgas=0.000 elapsed=7.74ms  mgasps=0.000 trieditt
[
  {
    caps: ["eth/65", "snap/1"],
    enode: "enode://37a3db1c1e34eac3611941ca5b7001168ac8eac28506649e6a5290979c7a78e7277565b37ec5d703871a9970a4794c51bde075d7991723256e0c07c8cb367eb8127.0.0.1:30303",
    id: "ab0dd2ac241e15eac929ff71bbb9d19f854213e77207f9ee0b74c4911801aef7",
    name: "geth/v1.13.14-stable-2bd6bd01/linux-amd64/go1.21.6",
    network: {
      inbound: false,
      localAddress: "127.0.0.1:60510",
      remoteAddress: "127.0.0.1:30303",
      static: true,
      trusted: false
    },
    protocols: {
      eth: {
        version: 65
      },
      snap: {
        version: 1
      }
    }
  }
]
```

Step 9: Send a transaction In Node 1 console, send transaction to Node 2

```
> eth.sendTransaction({
  .... from: "0xC8Fe5b129bb5e23eC4bD84CaB363ed92a2f9878",
  .... to: "0x41911D883221851DD8C0Fe7C2a3E623D507bBb19",
  .... value: "0xde0b6b3a7640000"
  .... })
INFO [04-08|18:03:16.769] Setting new local account      address=0xC8Fe5b129bb5e23eC4bD84CaB363ed92a2f9878
INFO [04-08|18:03:16.770] Submitted transaction      hash=0x6e201621cef61dc5e3cf114cbd03055c23b1f083ebf98411f8c91cb48e47265 from=0xC8Fe5b129bb5e23eC4bD84CaB363ed92a2f9878 nonce=
0 recipient=0x41911D883221851DD8C0Fe7C2a3E623D507bBb19 value=1,000,000,000,000,000
0x6e201621cef61dc5e3cf114cbd03055c23b1f083ebf98411f8c91cb48e47265
```

and check the balance of Node 2

```
> web3.fromWei(
  ...   eth.getBalance("0x41911D883221851DD8C0Fe7C2a3E623D507bBb19"),
  ...   "ether"
  ... )
101
```

CONCLUSION:

Through this experiment, two smart contracts - OrderRecord and ReviewVerification, were successfully created and deployed on the Ethereum Sepolia test network using Solidity and Remix IDE. The experiment demonstrated how blockchain technology can be integrated into a full-stack web application to permanently and transparently record order transactions and product reviews. The use of the Sepolia test network and MetaMask allowed safe testing without real funds. Overall, this experiment provided practical understanding of designing, compiling, deploying, and testing smart contracts on the Ethereum blockchain.